Formal Verification Report: Looputo Finance

Date	August 4th, 2026
Audit Repo	https://github.com/looputo-finance/Looputo-Finance
Audit Commit	`9c8a866`
Author	himanshu-Bhatt
Certora Prover version	certora-cli 8.19.2 (cloud, production server)
Compiler	solc 0.8.28, optimizer 200 runs, legacy pipeline (see Scope Assumptions)

Table of Contents

1. About Looputo
2. Formal Verification Methodology — Verification Approach · Types of Properties · Verification Process · Assumptions
3. Verification Properties — Libraries · LoopPool core · Aave v3 adapter · Morpho Blue adapter
4. Verification Results
5. Findings
6. Mutation Testing
7. Setup and Execution
8. Resources

1. About Looputo

Looputo Finance offers automated leveraged looping positions on yield-bearing collateral. A user deposits collateral (for example wstETH) or the debt asset (WETH); the pool flash-loans the debt asset from Morpho Blue, swaps it into collateral through an approved DEX aggregator, supplies the collateral to a lending market and borrows the debt asset to repay the flash loan, all in one transaction. Every user's stake in the pooled market position is tracked with two share counters, collateral shares and debt shares, priced against the pool's aggregate collateral and debt in the underlying market. A keeper bot can *deleverage* (repay debt, withdraw collateral) or *releverage* a set of positions when their target leverage drifts.

The contracts in scope:

1. **`LoopPool`** (abstract, 1 183 lines) — share accounting, the position lifecycle (`openPosition`, `addPositionLeverage`, `reducePositionLeverage`, `addPositionEquity`, `reducePositionEquity`, `modifyPositionRateSpread`, `closePosition`), the keeper operations (`deleverage`, `releverage`), the Morpho flash-loan callback, owner configuration and a custom reentrancy guard.

2. **`AaveV3LoopPool`** — Aave v3 adapter: supply/borrow/repay/withdraw against the Aave Pool, LTV checks via `getUserAccountData`, oracle price conversion through the Aave `PoolAddressesProvider`.
3. **`MorphoLoopPool`** — Morpho Blue adapter: supply/borrow/repay/withdraw against a single Morpho market, LTV checks recomputed with Morpho's virtual-shares arithmetic, share-based repayment.
4. **`SharesLib`, `MathLib`** — share ⇄ asset conversion with directional rounding, `mulDivDown` / `mulDivUp`.

`SparkLoopPool` is byte-for-byte identical to `AaveV3LoopPool` except for the provider address and type names; every Aave result transfers. `Factory`, `LoopPoolBeacon` and `BeaconTimelock` are thin wrappers around OpenZeppelin primitives that carry their own Certora specifications and are out of scope.

2. Formal Verification Methodology

Certora Formal Verification (FV) proves smart-contract correctness against a formal specification, examining all reachable states rather than the specific paths that tests and fuzzing sample. Properties are written in CVL (Certora Verification Language) and submitted with the compiled contracts to the prover, which turns bytecode and rules into a mathematical model and decides each rule's validity.

Verification Approach

Three scenes are verified, each with its own configuration file and harness:

Scene	Verified contract	Spec	What it proves
Libraries	`SharesLibHarness`	`certora/specs/SharesLib.spec`	The rounding primitives every share mint/burn relies on.
Aave	`AaveV3LoopPoolHarness`	`certora/specs/AaveV3LoopPool.spec` (imports `LoopPoolCore.spec`)	Core accounting + the Aave LTV/pricing adapter.
Morpho	`MorphoLoopPoolHarness`	`certora/specs/MorphoLoopPool.spec` (imports `LoopPoolCore.spec`)	Core accounting + the Morpho LTV/pricing/repay adapter.

`LoopPoolCore.spec` is protocol-agnostic and is verified twice, once in each adapter scene, so every core property holds against both market models. The harnesses (`certora/harness/*Harness.sol`) only add view getters that expose enum and struct fields as plain integers, a wrapper around the internal `_repayDebt` (Morpho) and wrappers that call the contract's own `_checkSystemLTV` / `_checkUserPositionLTV` predicates; they do not alter any production code path.

The external world is modelled by small Solidity mocks linked into the scene (`certora/harness/mocks/`): four OpenZeppelin ERC-20s (collateral, debt, aToken, variable-debt token), `MockAavePool` (1:1 aToken/vToken accounting, base-currency valuation with two unconstrained prices), `MockMorpho` (single market, Morpho's real virtual-shares math,

unconstrained interest injected on `accrueInterest`, a faithful no-fee `flashLoan`), the two oracles and an honest `MockAggregator`. The DEX router is the one genuinely unknown call: inside the flash-loan callback the low-level `aggregatorRouter.call(swapData)` is dispatched to the honest swap or to any token/market method, and otherwise summarised as `HAVOC_ECF`, meaning the callee may arbitrarily change the state of every contract except the pool itself. Every property below therefore holds *despite* a malicious router; conversely no property about token balances or solvency can be stated at all (see F-06).

Share conservation is tracked with `persistent ghost` sums maintained by storage hooks on `userPositions[u].collShares` / `.debtShares`.

Types of Properties

Valid-state invariants — inductive properties of storage that every external method preserves: share conservation, "Inactive ⇒ no shares", "Active ⇒ at least `MIN_SHARE_DELTA`", the reentrancy flag, and the aggregator allowlist never containing the market.

Access-control and revert properties — `@withrevert` rules confirming that owner-only setters, the keeper-only operations and the flash-loan callback reject every other caller.

Isolation and state-machine properties — parametric rules over every external method: a non-keeper call never touches another user's position; the position status only moves along the documented transitions; totals only move in position or keeper operations.

Unit / effect properties — before/after deltas of a single operation: what `openPosition`, `closePosition`, the four modify operations and `modifyPositionRateSpread` may and may not change, the cap, the `MIN_SHARE_DELTA` floors.

LTV properties — stated twice on purpose: once by re-evaluating the contract's own predicate on the post-state (catches a skipped check) and once by recomputing the bound in CVL from the mock's storage (catches a wrong predicate).

Keeper properties — the empty-array and exact-distribution behaviour of `deleverage` / `releverage`.

Reachability properties — `satisfy` rules proving that a reachable execution exists (owner power without bounds), plus `rule_sanity: basic` on every run, which fails a rule whose assertion is unreachable.

Verification Process

1. **Setup phase.** Mock the three market contracts, the oracles and the tokens; write the harnesses; link every address-typed storage field of the pool to its mock (`link` in the conf). Resolve the calls the prover cannot resolve statically: the flash-loan callback is dispatched back into the harness, SafeERC20's low-level token calls on a conditionally-selected token (`params.isColl ? COLL_TOKEN : DEBT_TOKEN`) and struct-encoded Morpho calls are dispatched optimistically to the scene's implementations, and only the router call

keeps the `HAVOC_ECF` default. This phase went through four iterations, each diagnosed from the prover's *Contracts Call Resolutions* panel (see Scope Assumptions).
2. **Crafting properties.** Write the core layer first (`LoopPoolCore.spec`), then the adapter layers as CVL recomputations of each adapter's LTV and pricing formulas.
3. **Non-vacuity.** Every rule runs with `rule_sanity: basic`; a rule whose assertion cannot be reached is reported as vacuous rather than verified. Parametric rules are additionally filtered to the verified contract (`f.contract == currentContract`) so that mock methods do not inflate the counts.
4. **Mutation testing.** One-line bugs are injected into `src/` and the property expected to catch each is re-run (section 6).

Assumptions

Assumptions address prover timeouts, tool limitations and state consistency, but incorrect ones can mask bugs by excluding reachable states. Each falls into one of five groups: **SAFE** (real-world constraints that do not reduce coverage), **SCOPE** (regime pins that fix which scenario a rule claims about, where the excluded region is covered elsewhere or immaterial), **UNSAFE** (tractability narrowings that exclude reachable, valid scenarios not covered by a sibling rule), **PROVED** (formally verified invariants reused as preconditions) and **TRUSTED** (initialization and off-chain facts assumed correct because they are excluded from verification).

Safe Assumptions

- **Environment (`setup(env e)`)** — the caller is neither the pool itself nor the flash-loan provider. Self-reentrancy is prevented by `nonReentrant`; Morpho only calls back into the pool's callback.
- **Tokens** — OpenZeppelin ERC-20 with 18 decimals, no fee-on-transfer, no rebasing (wstETH, weETH and WETH conform).
- **`MIN_SHARE_DELTA > 0`** in CORE-INV-04 and CORE-OP-04; a zero floor is a misconfiguration that the invariant is not meant to survive.

Scope Assumptions

- **Legacy compiler pipeline.** With `via_ir = true` the prover could not resolve the target of several SafeERC20 low-level calls (10 unresolved calls in `openPosition`, 10 in `addPositionEquity`, 4 in `closePosition`, 2 in `reducePositionEquity`; 0 in the leverage and keeper functions), each falling back to the adversarial `HAVOC_ECF` summary and producing spurious counterexamples. Verification therefore ran on the legacy pipeline. Solidity source semantics are identical; only bytecode shape differs. The library scene was run under both pipelines with identical results.
- **Loops** over `positions[]` in `deleverage` / `releverage` are unrolled twice (`loop_iter: 2`, `optimistic_loop`): results cover arrays of length 0, 1 and 2. The arithmetic in the loop body is length-independent.
- **Dynamic-length hashing** is optimistic (`optimistic_hashing`, `hashing_length_bound: 512`).
- **Prover budget** — the default SMT budget per rule and Certora's 2-hour global timeout per job. AAVE-LTV-03 / MORPHO-LTV-03 exceeded it in every run and is reported as not

verified; the Aave scene is split into two jobs to fit the budget.
- **Market models are more permissive than the real markets** (no health-factor / LLTV checks on borrow and withdraw, no Aave interest accrual). Every LTV property proven here therefore holds a fortiori against the real pool; an interest-accrual bug in the Aave adapter would not be detected.

Unsafe Assumptions

- **Aggregator model.** Outside the flash-loan callback every unresolved call is dispatched optimistically (`DISPATCH(optimistic=true)`) to a token or market mock. This is exactly right for SafeERC20 / Morpho calls whose real target is a known contract, but it also means a *non-callback* path that somehow reached an unknown contract would be pruned rather than havoced. The pool has no such path in the source.

Proved Assumptions

- **CORE-INV-03 (`inactiveHasNoShares`)** is reused as a precondition in the induction step of CORE-INV-01/02 and in CORE-OP-01 (`requireInvariant`), so `openPosition` starts from a well-formed Inactive slot.
- **CORE-INV-01/02** are reused as preconditions in CORE-KP-03/04.

Trusted Assumptions

- **Initialization.** Rules assume `isInitialized()`; invariants are inductive from the constructor state where initializers are disabled. The proxy's one-shot `initialize` via `BeaconProxy(beacon, initData)` is trusted (parametric instances on `initialize` and on the callback are reported as unreachable by the sanity check, as intended).
- **Oracles** return an unconstrained price; oracle manipulation resistance is not in scope.
- **The keeper is live** (F-05) and the owner's parameters are sane (F-04); both are documented rather than assumed away.

3. Verification Properties

Status legend: ✅ verified · ⏱ timed out / not run · ❌ violated (every ❌ maps to a finding in section 5) · — not run in this scene. Sub-text gives the number of parametric instances or the violating methods. Mutations: ❌ caught · ❓ not caught · ◌ not run.

Libraries (`SharesLib.spec`)

Property	Name	Description	Status	Mutations
SL-01 Rounding	`sharesUpGeqSharesDown`	Rounding up never returns fewer shares than rounding down. `toSharesUp >= toSharesDown`	✅ 1 instance(s) verified	

Property	Name	Description	Status	Mutations
SL-02 Rounding	`sharesUpMinusDownAtMostOne`	The two roundings differ by at most one share. `toSharesUp − toSharesDown <= 1`	✅ 1 instance(s) verified	
SL-03 Rounding	`assetsUpGeqAssetsDown`	Same for assets. `toAssetsUp >= toAssetsDown ∧ toAssetsUp − toAssetsDown <= 1`	✅ 1 instance(s) verified	
SL-04 Bootstrap	`bootstrapIsOneToOne`	With zero total shares, shares are minted 1:1 with assets. `totalShares == 0 => toShares(a) == a`	✅ 1 instance(s) verified	
SL-05 Bootstrap	`zeroTotalSharesGivesZeroAssets`	With zero total shares any share count is worth zero assets. `totalShares == 0 => toAssets(s) == 0`	✅ 1 instance(s) verified	
SL-06 Rounding	`roundTripAssetsDownNeverGains`	assets → sharesDown → assetsDown never returns more than the input. `toAssetsDown(toSharesDown(a)) <= a`	✅ 1 instance(s) verified	○ SharesLib.m1
SL-07 Monotonicity	`sharesDownMonotone`	toSharesDown is monotone in assets. `a1 <= a2 => toSharesDown(a1) <= toSharesDown(a2)`	✅ 1 instance(s) verified	
SL-08 Arithmetic	`mulDivUpIsCeil`	mulDivUp is a true ceiling. `mulDivUp(x,y,d) == ⌈x·y/d⌉`	✅ 1 instance(s) verified	○ MathLib.m1
SL-09 Arithmetic	`mulDivDownIsFloor`	mulDivDown is a true floor. `mulDivDown(x,y,d) == ⌊x·y/d⌋`	✅ 1 instance(s) verified	
SL-10 Arithmetic	`wMulDownBoundedByWad`	A fee computed with wMulDown never exceeds the base amount when the percentage is ≤ 1e18. `pct <= 1e18 => wMulDown(a, pct) <= a`	✅ 1 instance(s) verified	

LoopPool core (`LoopPoolCore.spec`, verified in both adapter scenes)

Property	Name	Description	Aave	Morpho	Mutations
CORE-INV-01 Valid state	`collSharesConserved`	Total collateral shares always equal the sum of every user's collateral shares. `totalCollShares == Σ_u userPositions[u].collShares` **Expected violation → F-01**	❌ violated on deleverage, releverage	❌ violated on deleverage, releverage	○ LoopPool.m1 ○ LoopPool.m6

Property	Name	Description	Aave	Morpho	Mutations
CORE-INV-02 Valid state	`debtSharesConserved`	Total debt shares always equal the sum of every user's debt shares. `totalDebtShares == Σ_u userPositions[u].debtShares` **Expected violation → F-01**	❌ violated on deleverage, releverage	❌ violated on deleverage, releverage	
CORE-INV-03 Valid state	`inactiveHasNoShares`	An Inactive position holds no shares. `status[u] == Inactive => collShares[u] == 0 && debtShares[u] == 0`	⏱ not verified: vacuous on addPositionEquity, openPosition; 20 other instance(s) verified	✅ 21 instance(s) verified	❌ LoopPool.m2
CORE-INV-04 Valid state	`activeHasShares`	An Active position holds at least MIN_SHARE_DELTA of each share (given MIN_SHARE_DELTA > 0). `status[u] == Active => collShares[u] >= MIN_SHARE_DELTA && debtShares[u] >= MIN_SHARE_DELTA` **Expected violation → F-03**	❌ violated on releverage	✅ 18 instance(s) verified	
CORE-INV-05 Valid state	`neverLeftEntered`	The reentrancy flag is never left in the ENTERED state after a call. `reentered != ENTERED`	⏱ not verified: vacuous on addPositionEquity, openPosition; 20 other instance(s) verified	✅ 23 instance(s) verified	○ LoopPool.m5
CORE-INV-06 Valid state	`marketNeverApprovedAggregator`	Neither the flash-loan provider nor the lending market is ever an approved aggregator. `!approvedAggregator[MORPHO] && !approvedAggregator[MARKET]`	⏱ not verified: vacuous on addPositionEquity, openPosition; 20 other instance(s) verified	— not run	
CORE-AC-01 Access control	`onlyOwnerChangesConfig`	Every configuration parameter (LTVs, cap, fees, fee collector, keeper, owner, aggregator allowlist) changes only when the owner calls. `config' != config => msg.sender == owner`	⏱ not verified: vacuous on addPositionEquity, openPosition; 17 other instance(s) verified	✅ 22 instance(s) verified	❌ LoopPool.m8

Property	Name	Description	Aave	Morpho	Mutations
CORE-AC-02 Access control	`onlyOwnerSyncsNetEquity`	Outside position and keeper operations, poolNetEquity changes only when the owner calls. `poolNetEquity' != poolNetEquity => msg.sender == owner`	✅ 11 instance(s) verified; initialize, onMorphoFlashLoan unreachable by construction	✅ 14 instance(s) verified	
CORE-AC-03 Access control	`onlyKeeperDeleverages`	deleverage rejects every caller except the keeper. `msg.sender != keeper => REVERTS`	✅ 1 instance(s) verified	✅ 1 instance(s) verified	❌ LoopPool.m3
CORE-AC-04 Access control	`onlyKeeperReleverages`	releverage rejects every caller except the keeper. `msg.sender != keeper => REVERTS`	✅ 1 instance(s) verified	✅ 1 instance(s) verified	❌ LoopPool.m3
CORE-AC-05 Access control	`callbackOnlyFromMorpho`	The flash-loan callback rejects every caller except MORPHO. `msg.sender != MORPHO => REVERTS`	✅ 1 instance(s) verified	✅ 1 instance(s) verified	○ LoopPool.m4
CORE-ISO-01 Isolation	`userIsolation`	A non-keeper call never changes another user's position (status, shares, leverage, spread). `u != msg.sender => userPositions[u]' == userPositions[u]`	⏱ not verified: vacuous on addPositionEquity, openPosition; 15 other instance(s) verified	✅ 20 instance(s) verified	
CORE-SM-01 State machine	`statusTransitions`	The only status transitions are: Inactive→Active by the user's own openPosition; any→Inactive by the user's own closePosition; any→Deleveraged/Unwound by deleverage; any→Active by releverage. `status' != status => (transition ∈ allowed set)`	⏱ not verified: vacuous on addPositionEquity, openPosition; 17 other instance(s) verified	✅ 22 instance(s) verified	

Property	Name	Description	Aave	Morpho	Mutations
CORE-SM-02 State machine	`keeperCannotTouchInactive`	Keeper operations never change the status of an Inactive position. `status[u] == Inactive => status[u]' == Inactive (for deleverage/releverage)` **Expected violation → F-03**	❌ violated on deleverage, releverage	❌ violated on deleverage, releverage	
CORE-SM-03 State machine	`totalSharesOnlyMoveInPositionOps`	Total shares change only in position or keeper operations. `totalCollShares' == totalCollShares && totalDebtShares' == totalDebtShares (other methods)`	✅ 11 instance(s) verified; initialize, onMorphoFlashLoan unreachable by construction	✅ 1 instance(s) verified	
CORE-OP-01 Unit	`openPositionEffects`	openPosition mints ≥ MIN_SHARE_DELTA of each share to the caller, credits the totals by exactly that amount, records leverage and spread, uses an approved aggregator and respects the cap. `status' == Active && c' >= MIN && totalColl' == totalColl + c' && poolNetEquity' <= cap && approvedAggregator[router]`	✅ 1 instance(s) verified	✅ 1 instance(s) verified	○ LoopPool.m1 ○ LoopPool.m7
CORE-OP-02 Unit	`closePositionEffects`	closePosition wipes the caller's position and debits the totals by exactly the former shares. `status' == Inactive && shares' == 0 && totalColl' == totalColl − c && totalDebt' == totalDebt − d`	✅ 1 instance(s) verified	✅ 1 instance(s) verified	❌ LoopPool.m2
CORE-OP-03 Unit	`addEquityRespectsCap`	addPositionEquity never breaches the cap. `poolNetEquity' <= cap`	✅ 1 instance(s) verified	✅ 1 instance(s) verified	○ LoopPool.m7
CORE-OP-04 Unit	`modifyOpsMonotone`	The two add operations strictly grow the caller's shares and the totals by the same amount; the two reduce operations strictly shrink them and never below MIN_SHARE_DELTA. `add: c' > c ∧ Δtotal == Δc ; reduce: c' < c ∧ c' >= MIN ∧ Δtotal == Δc`	⏱ not verified: vacuous on addPositionEquity; 3 other instance(s) verified	✅ 4 instance(s) verified	❌ LoopPool.m11
CORE-OP-05 Unit	`rateSpreadOnlyChangesSpread`	modifyPositionRateSpread changes nothing but the spread. `rateSpread' == s && (status, shares, leverage, totals) unchanged`	✅ 1 instance(s) verified	✅ 1 instance(s) verified	❌ LoopPool.m10
CORE-OP-06 Unit	`reduceEquityKeepsMinShares`	reducePositionEquity never leaves the caller below MIN_SHARE_DELTA. `c' >= MIN_SHARE_DELTA && d' >= MIN_SHARE_DELTA`	— not run	✅ 1 instance(s) verified	❌ LoopPool.m11

Property	Name	Description	Aave	Morpho	Mutations
CORE-KP-01 Keeper	`deleverageEmptyArrayKeepsTotals`	deleverage with an empty positions array must not change the totals (nobody is debited). `positions.length == 0 => totalColl' == totalColl && totalDebt' == totalDebt` **Expected violation → F-01**	❌ violated on rule	❌ violated on rule	
CORE-KP-02 Keeper	`releverageEmptyArrayKeepsTotals`	releverage with an empty positions array must not change the totals (nobody is credited). `positions.length == 0 => totalColl' == totalColl && totalDebt' == totalDebt` **Expected violation → F-02**	❌ violated on rule	❌ violated on rule	
CORE-KP-03 Keeper	`deleverageDistributesExactly`	With a non-empty positions array, deleverage debits users by exactly the burned totals. `positions.length > 0 ∧ conserved => conserved'`	✅ 1 instance(s) verified	✅ 1 instance(s) verified	○ LoopPool.m6
CORE-KP-04 Keeper	`releverageDistributesExactly`	With a non-empty positions array, releverage credits users by exactly the minted totals. `positions.length > 0 ∧ conserved => conserved'`	✅ 1 instance(s) verified	✅ 1 instance(s) verified	
CORE-LTV-01 LTV	`contractSystemLtvHoldsAfterFundsMovingOps`	After any funds-moving operation the contract's own system-LTV predicate (_checkSystemLTV) holds on the resulting state. `!revert(_checkSystemLTV()) after f`	⏱ not verified: vacuous on addPositionEquity, openPosition; 6 other instance(s) verified	✅ 8 instance(s) verified	❌ LoopPool.m9
CORE-LTV-02 LTV	`contractUserLtvHoldsAfterUserOps`	After open/modify operations the contract's own per-user LTV predicate holds for the caller. `!revert(_checkUserPositionLTV(msg.sender)) after f`	✅ 2 instance(s) verified	✅ 5 instance(s) verified	
CORE-LTV-03 LTV	`contractSystemLtvHoldsAfterOpen`	Single-method variant of CORE-LTV-01 for openPosition. `!revert(_checkSystemLTV()) after openPosition`	— not run	✅ 1 instance(s) verified	❌ LoopPool.m9
CORE-LTV-04 LTV	`contractUserLtvHoldsAfterDeleverageForNamed`	When deleverage names exactly one position, that position satisfies the per-user LTV predicate afterwards. `positions == [u] => !revert(_checkUserPositionLTV(u)) after deleverage`	— not run	✅ 1 instance(s) verified	○ LoopPool.m12

Property	Name	Description	Aave	Morpho	Mutations
CORE-OWN-01 Reachability	`ownerCanSetFeeAboveWad`	The owner can set an open fee above 100 % (no upper bound). `satisfy(openPositionFee' > 1e18)` **Expected violation → F-04**	✅ 1 instance(s) verified	✅ 1 instance(s) verified	
CORE-OWN-02 Reachability	`ownerCanSyncEquityAboveCap`	The owner can set poolNetEquity above the cap. `satisfy(poolNetEquity' > cap)` **Expected violation → F-04**	✅ 1 instance(s) verified	✅ 1 instance(s) verified	

Aave v3 adapter (`AaveV3LoopPool.spec`)

Property	Name	Description	Aave	Mutations
AAVE-LTV-01 LTV	`systemLtvHoldsAfterFundsMovingOps`	After any funds-moving operation the pool's Aave account satisfies collateral ≥ percentDivCeil(debt, systemTargetLTV), recomputed in CVL from the mock's storage. `collBase' >= [debtBase' · 1e4 / systemTargetLTV]`	— not run	○ AaveV3LoopPool.m1 ○ MorphoLoopPool.m2
AAVE-LTV-02 LTV	`userLtvHoldsAfterUserOps`	After open/modify operations the caller's share-weighted LTV (collateral rounded down, debt rounded up) is within positionMaxLTV. `toAssetsDown(c', TC', collBase') >= [toAssetsUp(d', TD', debtBase') · 1e4 / positionMaxLTV]`	⏱ not verified: vacuous on addPositionEquity, openPosition	○ AaveV3LoopPool.m2
AAVE-LTV-03 LTV	`closeRevertsWhenUserAboveMaxLtv_notRun`	closePosition refuses a caller whose position is above positionMaxLTV before the close. `userLtv > positionMaxLTV => REVERTS`	⏱ not run: exceeds the solver budget	
AAVE-LTV-04 LTV	`systemLtvFormulaAfterOpen`	Single-method variant of AAVE-LTV-01 for openPosition (mutation suite). `collBase' >= [debtBase' · 1e4 / systemTargetLTV] after openPosition`	⏱ not verified: vacuous on	○ AaveV3LoopPool.m1 ○ MorphoLoopPool.m2
AAVE-LTV-05 LTV	`userLtvFormulaAfterOpen`	Single-method variant of AAVE-LTV-02 for openPosition (mutation suite). `user LTV bound after openPosition`	⏱ not verified: vacuous on	○ AaveV3LoopPool.m2

Property	Name	Description	Aave	Mutations
AAVE-PX-01 Pricing	`convertCollToDebtMonotone`	convertCollToDebt is zero at zero and monotone in the amount. `f(0) == 0 ∧ a1 <= a2 => f(a1) <= f(a2)`	✅ 1 instance(s) verified	
AAVE-PX-02 Pricing	`convertCollToDebtFormula`	convertCollToDebt equals amount · collPrice · 10^dDec / (debtPrice · 10^cDec). `f(a) == a · P_coll · 10^18 / P_debt / 10^18`	✅ 1 instance(s) verified	○ AaveV3LoopPool.m3

Morpho Blue adapter (`MorphoLoopPool.spec`)

Property	Name	Description	Morpho	Mutations
MORPHO-LTV-01 LTV	`systemLtvHoldsAfterFundsMovingOps`	After any funds-moving operation the pool's Morpho position satisfies debt ≤ maxDebt(collateral, systemTargetLTV), recomputed in CVL with Morpho's virtual-shares math. `toAssetsUp(bs', TS'+1e6, TA'+1) <= ⌊⌊coll' · price / 1e36⌋ · systemTargetLTV / 1e18⌋`	✅ 8 instance(s) verified	○ AaveV3LoopPool.m1 ○ MorphoLoopPool.m2
MORPHO-LTV-02 LTV	`userLtvHoldsAfterUserOps`	After open/modify operations the caller's share-weighted debt is within positionMaxLTV. `toAssetsUp(d', TD', debt') <= maxDebt(toAssetsDown(c', TC', coll'), positionMaxLTV)`	✅ 4 instance(s) verified	○ AaveV3LoopPool.m2
MORPHO-LTV-03 LTV	`closeRevertsWhenUserAboveMaxLtv_notRun`	closePosition refuses a caller whose position is above positionMaxLTV before the close. `userDebt > maxDebt => REVERTS`	⏱ not run: exceeds the solver budget	
MORPHO-LTV-04 LTV	`systemLtvFormulaAfterOpen`	Single-method variant of MORPHO-LTV-01 for openPosition (mutation suite). `system LTV bound after openPosition`	⏱ no verdict on rule	○ AaveV3LoopPool.m1 ○ MorphoLoopPool.m2
MORPHO-PX-01 Pricing	`convertCollToDebtFormula`	convertCollToDebt equals amount · price / 1e36. `f(a) == ⌊a · price / 1e36⌋`	✅ 1 instance(s) verified	○ AaveV3LoopPool.m3
MORPHO-RP-01 Repay	`repayDebtClearsOrReduces`	_repayDebt never repays more shares than the pool holds: repaying at least the rounded-up outstanding debt clears the position, repaying less reduces it. `bs' <= bs ∧ (amount >= debtUp => bs' == 0)`	✅ 1 instance(s) verified	○ MorphoLoopPool.m3

Property	Name	Description	Morpho	Mutations
MORPHO-RP-02 Repay	`cachedDebtMatchesMorphoMath`	The cached collateral/debt used by every operation equals the Morpho-style rounded-up debt and raw collateral. `cachedDebt() == toAssetsUp(bs, TS+1e6, TA+1) ∧ cachedColl() == collateral`	✅ 1 instance(s) verified	○ MorphoLoopPool.m1

4. Verification Results

Group	Verified	Violated (finding)	Timeout / not run	Total
Libraries	10	0	0	10
LoopPool core — Aave scene	14	6	11	31
LoopPool core — Morpho scene	25	5	1	31
Aave v3 adapter	2	0	5	7
Morpho Blue adapter	5	0	2	7
Total	**56**	**11**	**19**	**86**

Every ❌ in the tables is an expected violation that documents a finding (section 5); no rule that was expected to hold is violated. Production runs on the Certora cloud prover:

Scene	Run
SharesLib / MathLib	a1f13cee…
AaveV3LoopPool, job A (invariants, effects, LTV)	3c980611…
AaveV3LoopPool, job B (parametric rules)	c1a47ffa…
MorphoLoopPool (all rules)	cc5b0ac4…

Where a round-6 job hit the global timeout before reaching a rule, the verdict of the previous identical run is used and recorded per cell in `audit/data/results.json`: Aave lean open-path job (wildcard dispatch); Morpho round 5 (streamed log, job 659ad55c); Mutation baseline, AaveV3LoopPool conf; Mutation baseline, LoopPool-core conf (Morpho scene); Mutation baseline, MorphoLoopPool conf.

5. Findings

Each finding below was produced by the Prover as a concrete counterexample against the audited commit. Severity follows the usual audit scale (Medium / Low / Informational). Findings F-01 to F-03 overlap with the manual PoCs in `test/KyberKeeperExploit.t.sol` and `test/KeeperExploit.t.sol`; the formal result adds the guarantee that **no other** code path breaks the corresponding property.

F-01 — `deleverage` with an empty `positions` array burns global shares that nobody owns

Severity	Medium
Rules	CORE-KP-01 `deleverageEmptyArrayKeepsTotals` (Violated, both scenes); CORE-INV-01/02 `collSharesConserved` / `debtSharesConserved` (induction step Violated on `deleverage`, both scenes)
Location	`src/LoopPool.sol`, `deleverage()` and `_distributeDeleveragedShares()`

`deleverage` first computes `totalDeleveragedCollShares` / `totalDeleveragedDebtShares` from the change in the pool's market position and subtracts them from `totalCollShares` / `totalDebtShares`. It then loops over `params.positions` to debit individual users. Nothing requires `params.positions.length > 0`, so with an empty array the loop is a no-op: the global totals shrink, no user is debited, and the per-user LTV check inside the loop is never executed.

After such a call `Σ userPositions[u].collShares > totalCollShares` (and likewise for debt). Because every user's claim is `collShares · totalCollAmount / totalCollShares`, the sum of all user claims now exceeds 100 % of the pool's collateral **and** debt. Combined with the aggregator's freedom to route swap proceeds anywhere (see `test/KyberKeeperExploit.t.sol`, `testF`), the keeper can extract value while leaving the accounting insolvent.

The Prover confirmed that with `positions.length ≥ 1` (CORE-KP-03 `deleverageDistributesExactly`, Verified) the distribution is exact, so the empty array is the only path that breaks conservation.

Recommendation. `require(params.positions.length > 0 && params.positions.length == params.collAmounts.length)`. Additionally require `Σ collAmounts[i] == params.swapAmount` so the last-element remainder cannot absorb a mismatch.

F-02 — `releverage` with an empty `positions` array mints global shares that nobody owns

Severity	Medium
Rules	CORE-KP-02 `releverageEmptyArrayKeepsTotals` (Violated, both scenes); CORE-INV-01/02 (induction step Violated on `releverage`)
Location	`src/LoopPool.sol`, `releverage()` and `_distributeReleverageShares()`

Symmetric to F-01: `totalCollShares` / `totalDebtShares` grow by the shares corresponding to the new borrow, but with an empty `positions` array nobody is credited. Every existing user's proportional claim on collateral *and* debt is diluted. The pool's Aave/Morpho account carries debt that no user position accounts for; it can only be repaid out of other users' collateral.

Recommendation. Same guard as F-01; require `Σ debtAmounts[i] == params.flashloanDebtAmount`.

F-03 — Keeper can change the status of a user who never opened a position

Severity	Low
Rules	CORE-SM-02 `keeperCannotTouchInactive` (Violated on `deleverage`, both scenes; Violated on `releverage` in the Morpho scene); CORE-INV-04 `activeHasShares` (induction step Violated on `releverage`)
Location	`src/LoopPool.sol`, `_distributeDeleveragedShares()` lines setting `position.status`, `_distributeReleverageShares()`

Neither distribution loop checks `position.status`. Passing an address with an `Inactive` position and a zero amount:

- in `deleverage` sets the status to `Deleveraged` / `Unwound` with zero shares. The address can no longer call `openPosition` (it requires `Inactive`) and can only escape via `closePosition`, which flash-loans `userDebtAmount = 0` and performs a zero swap;
- in `releverage` sets the status to `Active` with zero shares, violating the invariant that an Active position holds at least `MIN_SHARE_DELTA` of each share. Such a position passes every later `>= MIN_SHARE_DELTA` check only if the keeper first feeds it shares.

This is a griefing / state-corruption vector for a compromised or buggy keeper, not a direct theft.

Recommendation. In both loops, `require(position.status != PositionStatus.Inactive)`; in `_distributeReleverageShares` additionally require the position was `Deleveraged` or `Unwound`.

F-04 — Owner-settable parameters have no bounds

Severity	Low
Rules	CORE-OWN-01 `ownerCanSetFeeAboveWad` (reachable, both scenes); CORE-OWN-02 `ownerCanSyncEquityAboveCap` (reachable, both scenes); SL-10
Location	`src/LoopPool.sol`, `setFeeConfiguration`, `syncPoolNetEquity`, `setSystemTargetLTV`, `setPositionMaxLTV`

The Prover found executions in which the owner sets `openPositionFee > 1e18` (fee above 100 %, after which every `openPosition` reverts on `params.amount − fee` underflow, or, at exactly 1e18, forwards the entire deposit to `feeCollector`) and sets `poolNetEquity` above `cap` (after which no deposit can succeed until the owner lowers it). The same functions accept LTV targets above 100 %, which disables the on-chain LTV checks entirely.

These are consistent with the "Factory owner power is unbounded" note in `DEPLOY.md`. They are listed because the report would otherwise imply the fee and cap properties (CORE-OP-01, CORE-OP-03) hold unconditionally: they hold **given** sane owner parameters.

Recommendation. Bound `openPositionFee`, `closePositionFee` ≤ a protocol maximum (for example 5e16), `systemTargetLTV` / `positionMaxLTV` ≤ 1e4 (Aave/Spark) or ≤ 1e18 (Morpho), and place the owner behind the existing `BeaconTimelock`.

F-05 — `closePosition` is blocked for a user whose position exceeds `positionMaxLTV`

Severity	Informational
Rules	AAVE-LTV-03 / MORPHO-LTV-03 (`closeRevertsWhenUserAboveMaxLtv`): timed out in every run and is reported as **not verified**; the behaviour is evident from the source.
Location	`src/LoopPool.sol`, `closePosition()`, first statement `_checkUserPositionLTV(msg.sender)`

A user whose share-weighted LTV has drifted above `positionMaxLTV` (through price movement or interest) cannot close their own position until the keeper deleverages them. During that window the user depends entirely on keeper liveness. This is a design choice, but worth documenting for users.

F-06 — Aggregator swap proceeds are not tied to the pool (out of formal scope, confirmed by PoC)

Severity	Medium
Rules	none: modelled as assumption A4
Location	`src/LoopPool.sol`, `onMorphoFlashLoan()`

The audited commit has no `approvedAggregator` check inside the callback, and even with one the approved router lets the caller pick the swap recipient. The formal model deliberately treats the router as fully adversarial (`HAVOC_ECF`); every Verified property in section 3 therefore holds **despite** a malicious router, but no property about token balances or solvency can be stated at all. `test/KyberKeeperExploit.t.sol` demonstrates the drain concretely. Fixing F-01/F-02 does not fix this.

Recommendation. Restore the allowlist check in the callback, and enforce `receivedAmount >= oracleValue(input) · (1 − maxSlippage)` after every swap so proceeds must land in the pool.

Recommendations for the next verification round

1. **Re-run after the fixes.** The specs are written so that F-01/F-02/F-03 become Verified once the guards are added; `certora/confs/*.conf` can be run unchanged.
2. **Model the aggregator honestly once F-06 is fixed.** Replace the `HAVOC_ECF` default with `DISPATCH(optimistic=true)` to the honest mock plus an oracle-bounded slippage assumption, which unlocks solvency properties such as "pool net equity never decreases by more than the slippage bound in one keeper call".

3. **Interest accrual.** `MockMorpho` injects unconstrained interest on `accrueInterest`; `MockAavePool` accrues none. A rule stating that shares minted in `openPosition` are priced off post-accrual balances would close the remaining gap for the Aave adapter.
4. **`closeRevertsWhenUserAboveMaxLtv`.** Split the rule into a pure pre-check rule on a harness that exposes `_checkUserPositionLTV` (already present as `checkUserLtvExt`) to avoid symbolically executing the whole close path.
5. **Spark.** Run `AaveV3LoopPool.spec` against a `SparkLoopPoolHarness` for completeness; the diff is the provider address only.
6. **CI.** Add `certoraRun certora/confs/SharesLib.conf` to `.github/workflows/test.yml` (it finishes in under a minute) and the two adapter confs as a nightly job.

6. Mutation Testing

Mutation testing injects deliberate one-line bugs into the verified `src/` contracts and re-runs the properties expected to catch them, measuring how tightly the proofs constrain the implementation. A property that flips to a counterexample caught the bug. Mutants live under `certora/mutations/<Contract>/<Contract>.mN.sol` as full copies of the source file with a single `// MUTATION:` line; they are applied by `certoraMutate` in an isolated copy, never to the checked-out sources. Status icons per the legend.

Mutants	Caught	Not caught	Not run
20	6	0	14

Coverage by rule

Rule	Mutations
`openPositionEffects`	○ LoopPool.m1 ○ LoopPool.m7
`collSharesConserved`	○ LoopPool.m1 ○ LoopPool.m6
`closePositionEffects`	✗ LoopPool.m2
`inactiveHasNoShares`	✗ LoopPool.m2
`onlyKeeperDeleverages`	✗ LoopPool.m3
`onlyKeeperReleverages`	✗ LoopPool.m3
`callbackOnlyFromMorpho`	○ LoopPool.m4
`neverLeftEntered`	○ LoopPool.m5
`deleverageDistributesExactly`	○ LoopPool.m6
`addEquityRespectsCap`	○ LoopPool.m7
`onlyOwnerChangesConfig`	✗ LoopPool.m8
`contractSystemLtvHoldsAfterOpen`	✗ LoopPool.m9
`contractSystemLtvHoldsAfterFundsMovingOps`	✗ LoopPool.m9
`rateSpreadOnlyChangesSpread`	✗ LoopPool.m10
`reduceEquityKeepsMinShares`	✗ LoopPool.m11

Rule	Mutations
`modifyOpsMonotone`	❌ LoopPool.m11
`contractUserLtvHoldsAfterDeleverageForNamed`	○ LoopPool.m12
`systemLtvFormulaAfterOpen`	○ AaveV3LoopPool.m1 ○ MorphoLoopPool.m2
`systemLtvHoldsAfterFundsMovingOps`	○ AaveV3LoopPool.m1 ○ MorphoLoopPool.m2
`userLtvFormulaAfterOpen`	○ AaveV3LoopPool.m2
`userLtvHoldsAfterUserOps`	○ AaveV3LoopPool.m2
`convertCollToDebtFormula`	○ AaveV3LoopPool.m3
`cachedDebtMatchesMorphoMath`	○ MorphoLoopPool.m1
`repayDebtClearsOrReduces`	○ MorphoLoopPool.m3
`mulDivUpIsCeil`	○ MathLib.m1
`roundTripAssetsDownNeverGains`	○ SharesLib.m1

Mutants

Each entry gives the injected bug, the vulnerability class it models, the property expected to detect it and why, the rule's CVL source, the unified diff against the audited source, the verdict of the mutant run, and the commands to reproduce both the clean and the mutated run.

○ LoopPool.m1 — openPosition: drop `totalCollShares += mintedCollShares` (user minted shares not reflected in the total)

Mutant file	`certora/mutations/LoopPool/LoopPool.m1.sol`
Mutated source	`src/LoopPool.sol`
Bug class	Accounting desynchronisation: per-user shares minted without updating the global total.
Expected catcher	`openPositionEffects` (secondary: `collSharesConserved`)
Status	not run (prover budget exhausted, see section 4)

Why this rule catches it. CORE-OP-01 asserts `totalCollShares' == totalCollShares + collShares[msg.sender]'` after `openPosition`; with the total no longer credited the equality fails on the very first open. CORE-INV-01 (share conservation) would also break in its induction step for `openPosition`.

Diff against the audited source

```
--- a/src/LoopPool.sol
+++ b/src/LoopPool.sol
@@ -194,17 +194,17 @@
             cache.totalCollAmountBefore).toSharesDown(
                 cache.totalCollAmountBefore,
                 totalCollShares
             );
         require(
             mintedCollShares >= MIN_SHARE_DELTA,
```

```
            "Minted collShares cannot be less than MIN_SHARE_DELTA"
        );
-       totalCollShares += mintedCollShares;
+       // MUTATION: openPosition no longer credits the global collateral share
total

        uint256 mintedDebtShares = (cache.totalDebtAmountAfter -
            cache.totalDebtAmountBefore).toSharesUp(
                cache.totalDebtAmountBefore,
                totalDebtShares
            );
        require(
            mintedDebtShares >= MIN_SHARE_DELTA,
```

Property source (`certora/specs/LoopPoolCore.spec`)

```
/// CORE-OP-01: openPosition mints >= MIN_SHARE_DELTA of each share to the
/// caller, credits the totals by exactly that amount, records the requested
/// leverage/spread, uses an approved aggregator and respects the cap.
rule openPositionEffects(env e, ILoopPool.OpenParams p) {
    setup(e);
    requireInvariant inactiveHasNoShares(e.msg.sender);
    uint256 tc0 = totalCollShares();
    uint256 td0 = totalDebtShares();

    openPosition(e, p);

    uint256 c = positionCollShares(e.msg.sender);
    uint256 d = positionDebtShares(e.msg.sender);
    assert positionStatus(e.msg.sender) == ACTIVE();
    assert c >= MIN_SHARE_DELTA() && d >= MIN_SHARE_DELTA();
    assert to_mathint(totalCollShares()) == tc0 + c;
    assert to_mathint(totalDebtShares()) == td0 + d;
    assert positionTargetLeverage(e.msg.sender) == p.targetLeverage;
    assert positionRateSpread(e.msg.sender) == p.rateSpread;
    assert approvedAggregator(p.aggregatorRouter);
    assert poolNetEquity() <= cap();
}
```

Reproduce

```
# clean src/ -> VERIFIED
certoraRun certora/confs/mutation/LoopPool-core.conf --rule openPositionEffects
# with the mutation -> rule VIOLATED = mutant caught
certoraMutate certora/confs/mutation/LoopPool-core.conf
```

❌ LoopPool.m2 — closePosition: only reset status instead of deleting the position (stale shares survive close)

Mutant file	`certora/mutations/LoopPool/LoopPool.m2.sol`
Mutated source	`src/LoopPool.sol`
Bug class	Stale state on close: position marked Inactive while its shares survive.

Expected catcher	`closePositionEffects` (secondary: `inactiveHasNoShares`)
Status	caught (rule violated on the mutant)

Why this rule catches it. CORE-OP-02 asserts that after `closePosition` the caller's `collShares` and `debtShares` are zero. The mutant only resets the status, so the shares survive and the assertion fails; CORE-INV-03 (Inactive ⇒ no shares) breaks as well.

Diff against the audited source

```
--- a/src/LoopPool.sol
+++ b/src/LoopPool.sol
@@ -647,17 +647,17 @@

         require(
             params.swapAmount <= userCollAmount,
             "Required COLL_TOKEN exceeds available amount"
         );

         totalCollShares -= userPositions[msg.sender].collShares;
         totalDebtShares -= userPositions[msg.sender].debtShares;
-        delete userPositions[msg.sender];
+        userPositions[msg.sender].status = PositionStatus.Inactive; // MUTATION:
shares are not cleared on close

         uint256 debtTokenBalanceBefore = DEBT_TOKEN.balanceOf(address(this));

         _executeFlashloan(
             false,
             params.swapAmount,
             0,
             params.aggregatorRouter,
```

Property source (`certora/specs/LoopPoolCore.spec`)

```
/// CORE-OP-02: closePosition wipes the caller's position and debits the totals
/// by exactly the caller's former shares.
rule closePositionEffects(env e, ILoopPool.CloseParams p) {
    setup(e);
    uint256 c0 = positionCollShares(e.msg.sender);
    uint256 d0 = positionDebtShares(e.msg.sender);
    uint256 tc0 = totalCollShares();
    uint256 td0 = totalDebtShares();

    closePosition(e, p);

    assert positionStatus(e.msg.sender) == INACTIVE();
    assert positionCollShares(e.msg.sender) == 0 && positionDebtShares(e.msg.sender)
== 0;
    assert to_mathint(totalCollShares()) == tc0 - c0;
    assert to_mathint(totalDebtShares()) == td0 - d0;
}
```

Mutant run — every rule in the conf was re-run against the mutant; only the expected property flipped:

Rule	Verdict on the mutant
`addEquityRespectsCap`	✅ still verified
`callbackOnlyFromMorpho`	✅ still verified
`closePositionEffects`	❌ violated (mutant caught)
`contractSystemLtvHoldsAfterOpen`	✅ still verified
`contractUserLtvHoldsAfterDeleverageForNamed`	✅ still verified
`deleverageDistributesExactly`	✅ still verified
`neverLeftEntered`	✅ still verified
`onlyKeeperDeleverages`	✅ still verified
`onlyOwnerChangesConfig`	· unreachable instances only
`openPositionEffects`	✅ still verified
`rateSpreadOnlyChangesSpread`	✅ still verified
`reduceEquityKeepsMinShares`	✅ still verified

Run: https://prover.certora.com/output/1665037/fa15092ecb0a4d559151240af6565e29?anonymousKey=2e7567b7e0c972f153829af91c8009f45b478b42

Reproduce

```
# clean src/ -> VERIFIED
certoraRun certora/confs/mutation/LoopPool-core.conf --rule closePositionEffects
# with the mutation -> rule VIOLATED = mutant caught
certoraMutate certora/confs/mutation/LoopPool-core.conf
```

❌ LoopPool.m3 — onlyKeeper modifier: remove the keeper check (anyone can deleverage/releverage)

Mutant file	`certora/mutations/LoopPool/LoopPool.m3.sol`
Mutated source	`src/LoopPool.sol`
Bug class	Missing access control on a privileged operation.
Expected catcher	`onlyKeeperDeleverages` (secondary: `onlyKeeperReleverages`)
Status	caught (rule violated on the mutant)

Why this rule catches it. CORE-AC-03 calls `deleverage@withrevert` from an arbitrary sender and asserts `msg.sender != keeper ⇒ lastReverted`. Without the modifier's check a non-keeper call succeeds and the implication is falsified.

Diff against the audited source

```
--- a/src/LoopPool.sol
+++ b/src/LoopPool.sol
```

```
@@ -1174,12 +1174,12 @@
     modifier nonReentrant() {
         require(reentered != _ENTERED, "ReentrancyGuard: reentrant call");
         reentered = _ENTERED;
         _;
         reentered = _NOT_ENTERED;
     }

     modifier onlyKeeper() {
-        require(msg.sender == keeper, "Keeper bot only operation");
+        // MUTATION: keeper check removed
         _;
     }
 }
```

Property source (`certora/specs/LoopPoolCore.spec`)

```
/// CORE-AC-03: deleverage is keeper-only.
rule onlyKeeperDeleverages(env e, ILoopPool.DeleverageParams p) {
    setup(e);
    address k = keeper();
    deleverage@withrevert(e, p);
    assert e.msg.sender != k => lastReverted;
}
```

Mutant run — every rule in the conf was re-run against the mutant; only the expected property flipped:

Rule	Verdict on the mutant
`addEquityRespectsCap`	✅ still verified
`callbackOnlyFromMorpho`	✅ still verified
`closePositionEffects`	✅ still verified
`contractSystemLtvHoldsAfterOpen`	✅ still verified
`contractUserLtvHoldsAfterDeleverageForNamed`	✅ still verified
`deleverageDistributesExactly`	✅ still verified
`neverLeftEntered`	✅ still verified
`onlyKeeperDeleverages`	❌ violated (mutant caught)
`onlyOwnerChangesConfig`	· unreachable instances only
`openPositionEffects`	✅ still verified
`rateSpreadOnlyChangesSpread`	✅ still verified
`reduceEquityKeepsMinShares`	✅ still verified

Run: https://prover.certora.com/output/1665037/5c793d3899814230839f3d4bb498122d?anonymousKey=b96a3592da6bda5c1a90178b36137a99888f14af

Reproduce

```
# clean src/ -> VERIFIED
certoraRun certora/confs/mutation/LoopPool-core.conf --rule onlyKeeperDeleverages
# with the mutation -> rule VIOLATED = mutant caught
certoraMutate certora/confs/mutation/LoopPool-core.conf
```

○ LoopPool.m4 — onMorphoFlashLoan: remove the msg.sender == MORPHO check

Mutant file	`certora/mutations/LoopPool/LoopPool.m4.sol`
Mutated source	`src/LoopPool.sol`
Bug class	Missing caller check on the flash-loan callback (anyone can drive the pool's market position).
Expected catcher	`callbackOnlyFromMorpho`
Status	not run (prover budget exhausted, see section 4)

Why this rule catches it. CORE-AC-05 calls the callback from an arbitrary `msg.sender ≠ MORPHO` and expects a revert. With the check removed the callback runs (the router call is havoced, so a non-reverting path exists) and the rule is violated.

Diff against the audited source

```
--- a/src/LoopPool.sol
+++ b/src/LoopPool.sol
@@ -798,20 +798,17 @@
             totalDebtShares
         );
     }

     function onMorphoFlashLoan(
         uint256 assets,
         bytes calldata data
     ) external override {
-        require(
-            msg.sender == address(MORPHO),
-            "Flashloan callback can only triggered by morpho market"
-        );
+        // MUTATION: callback caller check removed

         // Decode the data
         // If add is true, "amount" is the minimum received amount of COLL_TOKEN
after swap
         // If add is false, "amount" is the input amount of COLL_TOKEN to swap
         (
             bool add,
             uint256 amount,
             uint256 extraAmount,
```

Property source (`certora/specs/LoopPoolCore.spec`)

```
/// CORE-AC-05: the flash-loan callback rejects every caller except MORPHO.
rule callbackOnlyFromMorpho(env e, uint256 assets, bytes data) {
    require isInitialized();
    address m = MORPHO();
```

```
    onMorphoFlashLoan@withrevert(e, assets, data);
    assert e.msg.sender != m => lastReverted;
}
```

Reproduce

```
# clean src/ -> VERIFIED
certoraRun certora/confs/mutation/LoopPool-core.conf --rule callbackOnlyFromMorpho
# with the mutation -> rule VIOLATED = mutant caught
certoraMutate certora/confs/mutation/LoopPool-core.conf
```

○ LoopPool.m5 — nonReentrant: never reset the flag after the call (pool bricks itself)

Mutant file	`certora/mutations/LoopPool/LoopPool.m5.sol`
Mutated source	`src/LoopPool.sol`
Bug class	Broken reentrancy guard (flag never reset; pool bricks itself after the first call).
Expected catcher	`neverLeftEntered`
Status	not run (prover budget exhausted, see section 4)

Why this rule catches it. CORE-INV-05 (`reentered != ENTERED`) is inductive: every guarded method must leave the flag reset. The mutant leaves it at ENTERED after any `nonReentrant` call, so the induction step fails on the first position operation.

Diff against the audited source

```
--- a/src/LoopPool.sol
+++ b/src/LoopPool.sol
@@ -1170,16 +1170,16 @@
     function _checkSystemAndUserPositionLTV(
         address user
     ) internal view virtual {}

     modifier nonReentrant() {
         require(reentered != _ENTERED, "ReentrancyGuard: reentrant call");
         reentered = _ENTERED;
         _;
-        reentered = _NOT_ENTERED;
+        // MUTATION: reentrancy flag never reset
     }

     modifier onlyKeeper() {
         require(msg.sender == keeper, "Keeper bot only operation");
         _;
     }
 }
```

Property source (`certora/specs/LoopPoolCore.spec`)

```
/// CORE-INV-05: the reentrancy flag is never left in the ENTERED state.
invariant neverLeftEntered()
    reentered() != ENTERED()
```

```
    filtered { f -> !f.isView && f.contract == currentContract }

/// CORE-INV-06: neither the flash-loan provider nor the lending market can be
/// an approved aggregator (they hold unlimited token approvals).
invariant marketNeverApprovedAggregator()
    !approvedAggregator(MORPHO()) && !approvedAggregator(MARKET())
    filtered { f -> !f.isView && f.contract == currentContract }
    { preserved with (env e) { require isInitialized(); } }

// ------------------------------------------------------------------------
// Access control
// ------------------------------------------------------------------------

/// CORE-AC-01: every configuration parameter changes only when the owner calls.
rule onlyOwnerChangesConfig(method f) filtered { f -> !f.isView && f.contract ==
currentContract } {
    env e; calldataarg args;
    setup(e);

    uint256 sLtv0 = systemTargetLTV();
    uint256 pLtv0 = positionMaxLTV();
    uint256 cap0 = cap();
    uint256 flFee0 = flashloanFeePercentage();
    uint256 oFee0 = openPositionFee();
    uint256 cFee0 = closePositionFee();
    address fc0 = feeCollector();
    address keeper0 = keeper();
    address owner0 = owner();
    address anyAgg;
    bool agg0 = approvedAggregator(anyAgg);

    f(e, args);

    bool configChanged =
        systemTargetLTV() != sLtv0 || positionMaxLTV() != pLtv0 || cap() != cap0 ||
        flashloanFeePercentage() != flFee0 || openPositionFee() != oFee0 ||
        closePositionFee() != cFee0 || feeCollector() != fc0 || keeper() != keeper0
||
        owner() != owner0 || approvedAggregator(anyAgg) != agg0;

    assert configChanged => e.msg.sender == owner0;
}
```

Reproduce

```
 # clean src/ -> VERIFIED
certoraRun certora/confs/mutation/LoopPool-core.conf --rule neverLeftEntered
# with the mutation -> rule VIOLATED = mutant caught
certoraMutate certora/confs/mutation/LoopPool-core.conf
```

○ LoopPool.m6 — _distributeDeleveragedShares: every element uses the rounded-down pro-rata share (remainder is burned from nobody)

Mutant file	`certora/mutations/LoopPool/LoopPool.m6.sol`

Mutated source	`src/LoopPool.sol`
Bug class	Rounding remainder lost in pro-rata distribution (shares burned from the total but from no user).
Expected catcher	`deleverageDistributesExactly` (secondary: `collSharesConserved`)
Status	not run (prover budget exhausted, see section 4)

Why this rule catches it. CORE-KP-03 assumes share conservation, runs `deleverage` with a non-empty array and asserts conservation afterwards. With every element rounded down, the sum of per-user debits is strictly less than the burned total whenever the division is inexact, so `Σ shares > totalShares` and the ghost equality fails.

Diff against the audited source

```
--- a/src/LoopPool.sol
+++ b/src/LoopPool.sol
@@ -1016,23 +1016,20 @@
         DeleverageCache memory cache
     ) internal {
         uint256 len = params.positions.length;

         uint256 alreadyDeleveragedCollShares;
         uint256 alreadyDeleveragedDebtShares;

         for (uint256 i; i < len; ) {
-            uint256 deleveragedCollShares = i != len - 1
-                ? params.collAmounts[i].mulDivDown(
+            uint256 deleveragedCollShares = params.collAmounts[i].mulDivDown( //
MUTATION: last element no longer absorbs the rounding remainder
                     cache.totalDeleveragedCollShares,
                     params.swapAmount
-                )
-                : cache.totalDeleveragedCollShares -
-                    alreadyDeleveragedCollShares;
+                );

             alreadyDeleveragedCollShares += deleveragedCollShares;

             uint256 deleveragedDebtShares = i != len - 1
                 ? params.collAmounts[i].mulDivDown(
                     cache.totalDeleveragedDebtShares,
                     params.swapAmount
                 )
```

Property source (`certora/specs/LoopPoolCore.spec`)

```
/// CORE-KP-03: with a non-empty positions array, deleverage distributes exactly
/// the burned shares (conservation restricted to the well-formed path).
rule deleverageDistributesExactly(env e, ILoopPool.DeleverageParams p) {
    setup(e);
    requireInvariant collSharesConserved();
    requireInvariant debtSharesConserved();
    require p.positions.length > 0;
    deleverage(e, p);
    assert to_mathint(totalCollShares()) == sumCollShares;
```

```
    assert to_mathint(totalDebtShares()) == sumDebtShares;
}
```

Reproduce

```
# clean src/ -> VERIFIED
certoraRun certora/confs/mutation/LoopPool-core.conf --rule
deleverageDistributesExactly
# with the mutation -> rule VIOLATED = mutant caught
certoraMutate certora/confs/mutation/LoopPool-core.conf
```

○ LoopPool.m7 — _increaseNetEquity: remove the cap check

Mutant file	`certora/mutations/LoopPool/LoopPool.m7.sol`
Mutated source	`src/LoopPool.sol`
Bug class	Missing deposit cap enforcement.
Expected catcher	`addEquityRespectsCap` (secondary: `openPositionEffects`)
Status	not run (prover budget exhausted, see section 4)

Why this rule catches it. CORE-OP-03 asserts `poolNetEquity' <= cap` after `addPositionEquity`; without the `require` a deposit that pushes equity past the cap succeeds. CORE-OP-01 has the same assertion for `openPosition`.

Diff against the audited source

```
--- a/src/LoopPool.sol
+++ b/src/LoopPool.sol
@@ -1118,17 +1118,17 @@
     function _increaseNetEquity(bool isColl, uint256 amount) internal {
         if (isColl) {
             uint256 equivalentAmount = convertCollToDebt(amount);
             poolNetEquity += equivalentAmount;
         } else {
             poolNetEquity += amount;
         }

-        require(poolNetEquity <= cap, "Deposit amount exceeds cap");
+        // MUTATION: cap check removed
     }

     function _decreaseNetEquity(uint256 amount) internal {
         if (amount > poolNetEquity) {
             poolNetEquity = 0;
         } else {
             poolNetEquity -= amount;
         }
```

Property source (`certora/specs/LoopPoolCore.spec`)

```
/// CORE-OP-03: adding equity never breaches the cap.
rule addEquityRespectsCap(env e, ILoopPool.ModifyEquityParams p) {
```

```
    setup(e);
    addPositionEquity(e, p);
    assert poolNetEquity() <= cap();
}
```

Reproduce

```
# clean src/ -> VERIFIED
certoraRun certora/confs/mutation/LoopPool-core.conf --rule addEquityRespectsCap
# with the mutation -> rule VIOLATED = mutant caught
certoraMutate certora/confs/mutation/LoopPool-core.conf
```

❌ LoopPool.m8 — setSystemTargetLTV: remove onlyOwner

Mutant file	`certora/mutations/LoopPool/LoopPool.m8.sol`
Mutated source	`src/LoopPool.sol`
Bug class	Missing access control on a configuration setter.
Expected catcher	`onlyOwnerChangesConfig`
Status	caught (rule violated on the mutant)

Why this rule catches it. CORE-AC-01 is parametric over every method and asserts that any change to a configuration field implies `msg.sender == owner`. The mutated setter lets any caller change `systemTargetLTV`, producing a counterexample on the `setSystemTargetLTV` instance.

Diff against the audited source

```
--- a/src/LoopPool.sol
+++ b/src/LoopPool.sol
@@ -852,17 +852,17 @@
             uint256 receivedAmount = debtTokenBalanceAfter -
                 debtTokenBalanceBefore;
             require(receivedAmount >= assets, "Swap exceeds defined slippage");
         }

         inputToken.forceApprove(aggregatorRouter, 0);
     }

-    function setSystemTargetLTV(uint256 newSystemTargetLTV) external onlyOwner {
+    function setSystemTargetLTV(uint256 newSystemTargetLTV) external { // MUTATION:
onlyOwner removed
         require(
             newSystemTargetLTV != systemTargetLTV,
             "System target LTV must be different from the current one"
         );
         systemTargetLTV = newSystemTargetLTV;
     }

     function setPositionMaxLTV(uint256 newPositionMaxLTV) external onlyOwner {
```

Property source (`certora/specs/LoopPoolCore.spec`)

```
/// CORE-AC-01: every configuration parameter changes only when the owner calls.
rule onlyOwnerChangesConfig(method f) filtered { f -> !f.isView && f.contract ==
currentContract } {
    env e; calldataarg args;
    setup(e);

    uint256 sLtv0 = systemTargetLTV();
    uint256 pLtv0 = positionMaxLTV();
    uint256 cap0 = cap();
    uint256 flFee0 = flashloanFeePercentage();
    uint256 oFee0 = openPositionFee();
    uint256 cFee0 = closePositionFee();
    address fc0 = feeCollector();
    address keeper0 = keeper();
    address owner0 = owner();
    address anyAgg;
    bool agg0 = approvedAggregator(anyAgg);

    f(e, args);

    bool configChanged =
        systemTargetLTV() != sLtv0 || positionMaxLTV() != pLtv0 || cap() != cap0 ||
        flashloanFeePercentage() != flFee0 || openPositionFee() != oFee0 ||
        closePositionFee() != cFee0 || feeCollector() != fc0 || keeper() != keeper0
||
        owner() != owner0 || approvedAggregator(anyAgg) != agg0;

    assert configChanged => e.msg.sender == owner0;
}
```

Mutant run — every rule in the conf was re-run against the mutant; only the expected property flipped:

Rule	Verdict on the mutant
`addEquityRespectsCap`	✅ still verified
`callbackOnlyFromMorpho`	✅ still verified
`closePositionEffects`	✅ still verified
`contractSystemLtvHoldsAfterOpen`	✅ still verified
`contractUserLtvHoldsAfterDeleverageForNamed`	✅ still verified
`deleverageDistributesExactly`	✅ still verified
`neverLeftEntered`	✅ still verified
`onlyKeeperDeleverages`	✅ still verified
`onlyOwnerChangesConfig`	❌ violated (mutant caught)
`openPositionEffects`	✅ still verified
`rateSpreadOnlyChangesSpread`	✅ still verified
`reduceEquityKeepsMinShares`	✅ still verified

Run: https://prover.certora.com/output/1665037/6bcedbfaacc940d0923b495910eeb34c?anonymousKey=81779f4b6c1916984075075312b4f211a1b8e9bc

Reproduce

```
# clean src/ -> VERIFIED
certoraRun certora/confs/mutation/LoopPool-core.conf --rule onlyOwnerChangesConfig
# with the mutation -> rule VIOLATED = mutant caught
certoraMutate certora/confs/mutation/LoopPool-core.conf
```

❌ LoopPool.m9 — openPosition: skip _checkSystemAndUserPositionLTV

Mutant file	`certora/mutations/LoopPool/LoopPool.m9.sol`
Mutated source	`src/LoopPool.sol`
Bug class	Skipped post-condition check: position opened without validating LTV.
Expected catcher	`contractSystemLtvHoldsAfterOpen` (secondary: `contractSystemLtvHoldsAfterFundsMovingOps`)
Status	caught (rule violated on the mutant)

Why this rule catches it. CORE-LTV-03 re-evaluates the contract's own `_checkSystemLTV()` predicate on the post-state of `openPosition` and asserts it does not revert. With the in-line check skipped, the prover finds an open that leaves the pool above the target LTV, and the harness predicate reverts.

Diff against the audited source

```
--- a/src/LoopPool.sol
+++ b/src/LoopPool.sol
@@ -215,17 +215,17 @@
         userPositions[msg.sender] = Position({
             status: PositionStatus.Active,
             targetLeverage: params.targetLeverage,
             rateSpread: params.rateSpread,
             collShares: mintedCollShares,
             debtShares: mintedDebtShares
         });

-        _checkSystemAndUserPositionLTV(msg.sender);
+        // MUTATION: LTV check skipped in openPosition

         emit PositionOpened(
             msg.sender,
             params.targetLeverage,
             params.rateSpread,
             mintedCollShares,
             mintedDebtShares
         );
```

Property source (`certora/specs/LoopPoolCore.spec`)

```
/// CORE-LTV-03: the contract's own system-LTV predicate holds after openPosition.
rule contractSystemLtvHoldsAfterOpen(env e, ILoopPool.OpenParams p) {
    setup(e);
```

```
    openPosition(e, p);
    checkSystemLtvExt@withrevert();
    assert !lastReverted;
}
```

Mutant run — every rule in the conf was re-run against the mutant; only the expected property flipped:

Rule	Verdict on the mutant
`addEquityRespectsCap`	✅ still verified
`callbackOnlyFromMorpho`	✅ still verified
`closePositionEffects`	✅ still verified
`contractSystemLtvHoldsAfterOpen`	❌ violated (mutant caught)
`contractUserLtvHoldsAfterDeleverageForNamed`	✅ still verified
`deleverageDistributesExactly`	✅ still verified
`neverLeftEntered`	✅ still verified
`onlyKeeperDeleverages`	✅ still verified
`onlyOwnerChangesConfig`	· unreachable instances only
`openPositionEffects`	✅ still verified
`rateSpreadOnlyChangesSpread`	✅ still verified
`reduceEquityKeepsMinShares`	✅ still verified

Run: https://prover.certora.com/output/1665037/1a3572df843a4a9bb1e51ebbcba0f437?anonymousKey=c83a3a7eadc147173e7fce1d3c2a014f999cec47

Reproduce

```
# clean src/ -> VERIFIED
certoraRun certora/confs/mutation/LoopPool-core.conf --rule
contractSystemLtvHoldsAfterOpen
# with the mutation -> rule VIOLATED = mutant caught
certoraMutate certora/confs/mutation/LoopPool-core.conf
```

❌ LoopPool.m10 — modifyPositionRateSpread: also overwrite targetLeverage

Mutant file	`certora/mutations/LoopPool/LoopPool.m10.sol`
Mutated source	`src/LoopPool.sol`
Bug class	Unintended state write: a spread update also overwrites the target leverage.
Expected catcher	`rateSpreadOnlyChangesSpread`
Status	caught (rule violated on the mutant)

Why this rule catches it. CORE-OP-05 asserts that `targetLeverage` is unchanged by `modifyPositionRateSpread`; the mutant overwrites it with the new spread.

Diff against the audited source

```
--- a/src/LoopPool.sol
+++ b/src/LoopPool.sol
@@ -606,16 +606,17 @@

         uint64 oldRateSpread = userPositions[msg.sender].rateSpread;
         require(
             oldRateSpread != newRateSpread,
             "New rate spread cannot be same as the current spread"
         );

         userPositions[msg.sender].rateSpread = newRateSpread;
+        userPositions[msg.sender].targetLeverage = newRateSpread; // MUTATION: also
overwrites targetLeverage

         emit PositionRateSpreadModified(
             msg.sender,
             oldRateSpread,
             newRateSpread
         );
     }
```

Property source (`certora/specs/LoopPoolCore.spec`)

```
/// CORE-OP-05: modifyPositionRateSpread changes nothing but the spread.
rule rateSpreadOnlyChangesSpread(env e, uint64 s) {
    setup(e);
    uint8 st0 = positionStatus(e.msg.sender);
    uint256 c0 = positionCollShares(e.msg.sender);
    uint256 d0 = positionDebtShares(e.msg.sender);
    uint64 lev0 = positionTargetLeverage(e.msg.sender);
    uint256 tc0 = totalCollShares();
    uint256 td0 = totalDebtShares();

    modifyPositionRateSpread(e, s);

    assert positionRateSpread(e.msg.sender) == s;
    assert positionStatus(e.msg.sender) == st0 && positionCollShares(e.msg.sender)
== c0 &&
           positionDebtShares(e.msg.sender) == d0 &&
positionTargetLeverage(e.msg.sender) == lev0;
    assert totalCollShares() == tc0 && totalDebtShares() == td0;
}
```

Mutant run — every rule in the conf was re-run against the mutant; only the expected property flipped:

Rule	Verdict on the mutant
`addEquityRespectsCap`	✅ still verified
`callbackOnlyFromMorpho`	✅ still verified
`closePositionEffects`	✅ still verified
`contractSystemLtvHoldsAfterOpen`	✅ still verified

Rule	Verdict on the mutant
`contractUserLtvHoldsAfterDeleverageForNamed`	✅ still verified
`deleverageDistributesExactly`	✅ still verified
`neverLeftEntered`	✅ still verified
`onlyKeeperDeleverages`	✅ still verified
`onlyOwnerChangesConfig`	· unreachable instances only
`openPositionEffects`	✅ still verified
`rateSpreadOnlyChangesSpread`	❌ violated (mutant caught)
`reduceEquityKeepsMinShares`	✅ still verified

Run: https://prover.certora.com/output/1665037/743d701fd33e4d389e71046d6927bb52?anonymousKey=2383e3103d65949ed01da1307b21c67307d9c853

Reproduce

```
# clean src/ -> VERIFIED
certoraRun certora/confs/mutation/LoopPool-core.conf --rule
rateSpreadOnlyChangesSpread
# with the mutation -> rule VIOLATED = mutant caught
certoraMutate certora/confs/mutation/LoopPool-core.conf
```

❌ LoopPool.m11 — reducePositionEquity: drop the `debtShares >= MIN_SHARE_DELTA` floor

Mutant file	`certora/mutations/LoopPool/LoopPool.m11.sol`
Mutated source	`src/LoopPool.sol`
Bug class	Missing dust floor: a position can be reduced to unprocessable dust.
Expected catcher	`reduceEquityKeepsMinShares` (secondary: `modifyOpsMonotone`)
Status	caught (rule violated on the mutant)

Why this rule catches it. CORE-OP-06 asserts `debtShares' >= MIN_SHARE_DELTA` after `reducePositionEquity`; with the floor removed a reduction can leave 1 wei of debt shares.

Diff against the audited source

```
--- a/src/LoopPool.sol
+++ b/src/LoopPool.sol
@@ -568,20 +568,17 @@
                 totalDebtShares
             );
         require(
             burnedDebtShares >= MIN_SHARE_DELTA,
             "Burned debtShares cannot be less than MIN_SHARE_DELTA"
         );

         userPosition.debtShares -= burnedDebtShares;
-        require(
-            userPosition.debtShares >= MIN_SHARE_DELTA,
```

```
-            "User debtShares cannot be less than MIN_SHARE_DELTA"
-        );
+        // MUTATION: reducePositionEquity may leave a dust debt position
         totalDebtShares -= burnedDebtShares;

         _checkSystemAndUserPositionLTV(msg.sender);

         uint256 fee = withdrawableEquity.wMulDown(closePositionFee);
         DEBT_TOKEN.safeTransfer(msg.sender, withdrawableEquity - fee);
         if (fee > 0) {
             DEBT_TOKEN.safeTransfer(feeCollector, fee);
```

Property source (`certora/specs/LoopPoolCore.spec`)

```
/// CORE-OP-06: reducePositionEquity never leaves the caller below MIN_SHARE_DELTA.
rule reduceEquityKeepsMinShares(env e, ILoopPool.ModifyEquityParams p) {
    setup(e);
    reducePositionEquity(e, p);
    assert positionCollShares(e.msg.sender) >= MIN_SHARE_DELTA();
    assert positionDebtShares(e.msg.sender) >= MIN_SHARE_DELTA();
}
```

Mutant run — every rule in the conf was re-run against the mutant; only the expected property flipped:

Rule	Verdict on the mutant
`addEquityRespectsCap`	✅ still verified
`callbackOnlyFromMorpho`	✅ still verified
`closePositionEffects`	✅ still verified
`contractSystemLtvHoldsAfterOpen`	✅ still verified
`contractUserLtvHoldsAfterDeleverageForNamed`	✅ still verified
`deleverageDistributesExactly`	✅ still verified
`neverLeftEntered`	✅ still verified
`onlyKeeperDeleverages`	✅ still verified
`onlyOwnerChangesConfig`	· unreachable instances only
`openPositionEffects`	✅ still verified
`rateSpreadOnlyChangesSpread`	✅ still verified
`reduceEquityKeepsMinShares`	❌ violated (mutant caught)

Run: https://prover.certora.com/output/1665037/6bb9394fadf84094909f42eacf49c519?anonymousKey=502dcfdffe76867ae0b2df0f76d024d3435c7a0d

Reproduce

```
# clean src/ -> VERIFIED
certoraRun certora/confs/mutation/LoopPool-core.conf --rule
reduceEquityKeepsMinShares
```

```
# with the mutation -> rule VIOLATED = mutant caught
certoraMutate certora/confs/mutation/LoopPool-core.conf
```

○ LoopPool.m12 — _distributeDeleveragedShares: skip the per-user LTV check

Mutant file	`certora/mutations/LoopPool/LoopPool.m12.sol`
Mutated source	`src/LoopPool.sol`
Bug class	Skipped per-user post-condition after a keeper deleverage.
Expected catcher	`contractUserLtvHoldsAfterDeleverageForNamed`
Status	not run (prover budget exhausted, see section 4)

Why this rule catches it. CORE-LTV-04 runs `deleverage` naming exactly one position and re-evaluates `_checkUserPositionLTV` for it afterwards. Without the in-loop check a keeper can leave the named position above `positionMaxLTV`, and the harness predicate reverts.

Diff against the audited source

```
--- a/src/LoopPool.sol
+++ b/src/LoopPool.sol
@@ -1049,17 +1049,17 @@

             position.collShares = newCollShares;
             position.debtShares = newDebtShares;

             position.status = params.status == PositionStatus.Unwound
                 ? PositionStatus.Unwound
                 : PositionStatus.Deleveraged;

-            _checkUserPositionLTV(user);
+            // MUTATION: per-user LTV check skipped after deleverage

             emit PositionDeleveraged(user, newCollShares, newDebtShares);

             unchecked {
                 ++i;
             }
         }
     }
```

Property source (`certora/specs/LoopPoolCore.spec`)

```
/// CORE-LTV-04: after deleverage names exactly one position, that position
/// satisfies the contract's own per-user LTV predicate.
rule contractUserLtvHoldsAfterDeleverageForNamed(env e, ILoopPool.DeleverageParams
p) {
    setup(e);
    require p.positions.length == 1;
    address u = p.positions[0];
    deleverage(e, p);
    checkUserLtvExt@withrevert(u);
```

```
    assert !lastReverted;
}
```

Reproduce

```
# clean src/ -> VERIFIED
certoraRun certora/confs/mutation/LoopPool-core.conf --rule
contractUserLtvHoldsAfterDeleverageForNamed
# with the mutation -> rule VIOLATED = mutant caught
certoraMutate certora/confs/mutation/LoopPool-core.conf
```

○ AaveV3LoopPool.m1 — _checkSystemAndUserPositionLTV: system check rounds the debt requirement down (percentDiv) instead of up

Mutant file	`certora/mutations/AaveV3LoopPool/AaveV3LoopPool.m1.sol`
Mutated source	`src/AaveV3LoopPool.sol`
Bug class	Rounding in the protocol's disfavour in the system LTV check (floor instead of ceiling).
Expected catcher	`systemLtvFormulaAfterOpen` (secondary: `systemLtvHoldsAfterFundsMovingOps`)
Status	not run (prover budget exhausted, see section 4)

Why this rule catches it. AAVE-LTV-04 recomputes `collBase >= [debtBase·1e4/systemTargetLTV]` in CVL after `openPosition`. The mutated contract check uses the floor, so a state where `collBase` is between the floor and the ceiling passes the contract but violates the CVL bound.

Diff against the audited source

```
--- a/src/AaveV3LoopPool.sol
+++ b/src/AaveV3LoopPool.sol
@@ -318,17 +318,17 @@
             ,
             ,
             ,

         ) = IPool(MARKET).getUserAccountData(address(this));

         require(
             collateralInBaseCurrency >=
-                debtInBaseCurrency.percentDivCeil(systemTargetLTV),
+                debtInBaseCurrency.percentDiv(systemTargetLTV), // MUTATION: floor
instead of ceil
             "System LTV must be less than the system target LTV"
         );

         uint256 userCollShares = userPositions[user].collShares;
         uint256 userDebtShares = userPositions[user].debtShares;
         uint256 userCollateralInBaseCurrency = userCollShares.toAssetsDown(
             totalCollShares,
             collateralInBaseCurrency
```

Property source (`certora/specs/AaveV3LoopPool.spec`)

```
/// AAVE-LTV-04: single-method variant of AAVE-LTV-01 for openPosition.
rule systemLtvFormulaAfterOpen(env e, ILoopPool.OpenParams p) {
    setup(e);
    openPosition(e, p);
    mathint ltv = to_mathint(systemTargetLTV());
    assert ltv > 0;
    assert systemColl() >= percentDivCeil(systemDebt(), ltv);
}
```

Reproduce

```
# clean src/ -> VERIFIED
certoraRun certora/confs/mutation/AaveV3LoopPool.conf --rule
systemLtvFormulaAfterOpen
# with the mutation -> rule VIOLATED = mutant caught
certoraMutate certora/confs/mutation/AaveV3LoopPool.conf
```

○ AaveV3LoopPool.m2 — _checkSystemAndUserPositionLTV: user debt rounded down (toAssetsDown) instead of up

Mutant file	`certora/mutations/AaveV3LoopPool/AaveV3LoopPool.m2.sol`
Mutated source	`src/AaveV3LoopPool.sol`
Bug class	Rounding in the user's favour when valuing the user's debt.
Expected catcher	`userLtvFormulaAfterOpen` (secondary: `userLtvHoldsAfterUserOps`)
Status	not run (prover budget exhausted, see section 4)

Why this rule catches it. AAVE-LTV-05 values the user's debt with `toAssetsUp` (rounded against the user) in CVL. The mutant rounds it down inside the contract, admitting a position whose rounded-up debt exceeds the bound.

Diff against the audited source

```
--- a/src/AaveV3LoopPool.sol
+++ b/src/AaveV3LoopPool.sol
@@ -328,17 +328,17 @@
         );

         uint256 userCollShares = userPositions[user].collShares;
         uint256 userDebtShares = userPositions[user].debtShares;
         uint256 userCollateralInBaseCurrency = userCollShares.toAssetsDown(
             totalCollShares,
             collateralInBaseCurrency
         );
-        uint256 userDebtInBaseCurrency = userDebtShares.toAssetsUp(
+        uint256 userDebtInBaseCurrency = userDebtShares.toAssetsDown( // MUTATION:
user debt rounded down
             totalDebtShares,
             debtInBaseCurrency
         );
```

```
        require(
            userCollateralInBaseCurrency >=
                userDebtInBaseCurrency.percentDivCeil(positionMaxLTV),
            "User Position LTV must be less than the position maximum LTV"
```

Property source (`certora/specs/AaveV3LoopPool.spec`)

```
/// AAVE-LTV-05: single-method variant of AAVE-LTV-02 for openPosition.
rule userLtvFormulaAfterOpen(env e, ILoopPool.OpenParams p) {
    setup(e);
    openPosition(e, p);
    mathint pltv = to_mathint(positionMaxLTV());
    mathint uc = toAssetsDown(positionCollShares(e.msg.sender), totalCollShares(),
systemColl());
    mathint ud = toAssetsUp(positionDebtShares(e.msg.sender), totalDebtShares(),
systemDebt());
    assert pltv > 0;
    assert uc >= percentDivCeil(ud, pltv);
}
```

Reproduce

```
# clean src/ -> VERIFIED
certoraRun certora/confs/mutation/AaveV3LoopPool.conf --rule userLtvFormulaAfterOpen
# with the mutation -> rule VIOLATED = mutant caught
certoraMutate certora/confs/mutation/AaveV3LoopPool.conf
```

○ AaveV3LoopPool.m3 — convertCollToDebt: collateral and debt prices swapped

Mutant file	`certora/mutations/AaveV3LoopPool/AaveV3LoopPool.m3.sol`
Mutated source	`src/AaveV3LoopPool.sol`
Bug class	Price inversion in the collateral-to-debt conversion used by the cap check.
Expected catcher	`convertCollToDebtFormula`
Status	not run (prover budget exhausted, see section 4)

Why this rule catches it. AAVE-PX-02 pins `convertCollToDebt(a)` to `a·P_coll·10^18/P_debt/10^18`; swapping the prices changes the result whenever `P_coll ≠ P_debt`.

Diff against the audited source

```
--- a/src/AaveV3LoopPool.sol
+++ b/src/AaveV3LoopPool.sol
@@ -67,19 +67,19 @@
             poolAddressesProvider.getPriceOracle()
         );

         uint256 collateralPrice = oracle.getAssetPrice(address(COLL_TOKEN));
         uint256 debtPrice = oracle.getAssetPrice(address(DEBT_TOKEN));
```

```
         amountInDebt =
             (amount *
-                collateralPrice *
+                debtPrice * // MUTATION: prices swapped
                 (10 ** ERC20(address(DEBT_TOKEN)).decimals())) /
-            debtPrice /
+            collateralPrice /
             (10 ** ERC20(address(COLL_TOKEN)).decimals());
     }

     function supplyRateView(
         uint128 addedSupplyAmount
     ) public view override returns (uint256) {
         DataTypes.ReserveData memory reserve = IPool(MARKET).getReserveData(
             address(COLL_TOKEN)
```

Property source (`certora/specs/AaveV3LoopPool.spec`)

```
/// AAVE-PX-02: convertCollToDebt scales linearly with the oracle ratio
/// (documents the formula: amount * collPrice * 10^dDec / (debtPrice * 10^cDec)).
rule convertCollToDebtFormula(uint256 amount) {
    mathint cp = to_mathint(aaveOracle.getAssetPrice(COLL_TOKEN()));
    mathint dp = to_mathint(aaveOracle.getAssetPrice(DEBT_TOKEN()));
    require dp > 0;
    // MockERC20 has 18 decimals on both sides.
    assert to_mathint(convertCollToDebt(amount)) == (amount * cp * 10^18) / dp /
10^18;
}
```

Reproduce

```
# clean src/ -> VERIFIED
certoraRun certora/confs/mutation/AaveV3LoopPool.conf --rule
convertCollToDebtFormula
# with the mutation -> rule VIOLATED = mutant caught
certoraMutate certora/confs/mutation/AaveV3LoopPool.conf
```

○ MorphoLoopPool.m1 — _cacheCollAndDebt: pool debt rounded down instead of up

Mutant file	`certora/mutations/MorphoLoopPool/MorphoLoopPool.m1.sol`
Mutated source	`src/MorphoLoopPool.sol`
Bug class	Pool debt under-estimated (rounded down) in the cached market view.
Expected catcher	`cachedDebtMatchesMorphoMath`
Status	not run (prover budget exhausted, see section 4)

Why this rule catches it. MORPHO-RP-02 asserts `cachedDebt() == toAssetsUp(borrowShares, totalShares+1e6, totalAssets+1)`; rounding down differs by one unit whenever the division is inexact.

Diff against the audited source

```
--- a/src/MorphoLoopPool.sol
+++ b/src/MorphoLoopPool.sol
@@ -82,17 +82,17 @@
         returns (uint256 collAmount, uint256 debtAmount)
     {
         MorphoPosition memory position = IMorpho(MARKET).position(
             ID,
             address(this)
         );
         MorphoMarket memory market = IMorpho(MARKET).market(ID);
         collAmount = uint256(position.collateral);
-        debtAmount = uint256(position.borrowShares).toAssetsUp(
+        debtAmount = uint256(position.borrowShares).toAssetsDown( // MUTATION: pool debt rounded down
             market.totalBorrowShares + VIRTUAL_SHARES,
             market.totalBorrowAssets + VIRTUAL_ASSETS
         );
     }

     function _supplyCollateral(uint256 amounts) internal override {
         IMorpho(MARKET).supplyCollateral(
             MARKET_PARAMS,
```

Property source (`certora/specs/MorphoLoopPool.spec`)

```
/// MORPHO-RP-02: the harness's cached debt equals the Morpho-style rounded-up
/// debt used by the LTV checks (the two code paths agree).
rule cachedDebtMatchesMorphoMath() {
    assert to_mathint(cachedDebt()) == systemDebt();
    assert to_mathint(cachedColl()) == systemColl();
}
```

Reproduce

```
# clean src/ -> VERIFIED
certoraRun certora/confs/mutation/MorphoLoopPool.conf --rule cachedDebtMatchesMorphoMath
# with the mutation -> rule VIOLATED = mutant caught
certoraMutate certora/confs/mutation/MorphoLoopPool.conf
```

○ MorphoLoopPool.m2 — _checkSystemAndUserPositionLTV: system max debt ignores systemTargetLTV (allows up to 100% LTV)

Mutant file	`certora/mutations/MorphoLoopPool/MorphoLoopPool.m2.sol`
Mutated source	`src/MorphoLoopPool.sol`
Bug class	LTV parameter ignored: the system check allows borrowing up to 100 % of collateral value.
Expected catcher	`systemLtvFormulaAfterOpen` (secondary: `systemLtvHoldsAfterFundsMovingOps`)
Status	not run (prover budget exhausted, see section 4)

Why this rule catches it. MORPHO-LTV-04 asserts `debt <= [[coll·price/1e36] ·systemTargetLTV/1e18]` after `openPosition`; the mutant compares against `coll·price/1e36` alone, admitting any state with `systemTargetLTV < 1e18`.

Diff against the audited source

```
--- a/src/MorphoLoopPool.sol
+++ b/src/MorphoLoopPool.sol
@@ -216,18 +216,17 @@
         uint256 collateralPrice = IOracle(MARKET_PARAMS.oracle).price();

         uint256 collAmount = uint256(position.collateral);
         uint256 debtAmount = uint256(position.borrowShares).toAssetsUp(
             market.totalBorrowShares + VIRTUAL_SHARES,
             market.totalBorrowAssets + VIRTUAL_ASSETS
         );
         uint256 maxDebtAmount = collAmount
-            .mulDivDown(collateralPrice, ORACLE_PRICE_SCALE)
-            .wMulDown(systemTargetLTV);
+            .mulDivDown(collateralPrice, ORACLE_PRICE_SCALE); // MUTATION:
systemTargetLTV factor dropped

         require(
             maxDebtAmount >= debtAmount,
             "System LTV must be less than the system target LTV"
         );

         uint256 userCollShares = userPositions[user].collShares;
         uint256 userDebtShares = userPositions[user].debtShares;
```

Property source (`certora/specs/AaveV3LoopPool.spec`)

```
/// AAVE-LTV-04: single-method variant of AAVE-LTV-01 for openPosition.
rule systemLtvFormulaAfterOpen(env e, ILoopPool.OpenParams p) {
    setup(e);
    openPosition(e, p);
    mathint ltv = to_mathint(systemTargetLTV());
    assert ltv > 0;
    assert systemColl() >= percentDivCeil(systemDebt(), ltv);
}
```

Reproduce

```
# clean src/ -> VERIFIED
certoraRun certora/confs/mutation/MorphoLoopPool.conf --rule
systemLtvFormulaAfterOpen
# with the mutation -> rule VIOLATED = mutant caught
certoraMutate certora/confs/mutation/MorphoLoopPool.conf
```

○ MorphoLoopPool.m3 — _repayDebt: the clamp branch repays only half of the pool's borrow shares

Mutant file	`certora/mutations/MorphoLoopPool/MorphoLoopPool.m3.sol`
Mutated source	`src/MorphoLoopPool.sol`
Bug class	Incomplete repayment when clamping to the pool's outstanding shares.
Expected catcher	`repayDebtClearsOrReduces`
Status	not run (prover budget exhausted, see section 4)

Why this rule catches it. MORPHO-RP-01 asserts that repaying at least the rounded-up outstanding debt clears the pool's borrow shares; repaying only half of the shares leaves `borrowShares' > 0`.

Diff against the audited source

```
--- a/src/MorphoLoopPool.sol
+++ b/src/MorphoLoopPool.sol
@@ -130,17 +130,17 @@
         bool repayInAmounts;

         uint256 shares = amounts.toSharesDown(
             market.totalBorrowAssets + VIRTUAL_ASSETS,
             market.totalBorrowShares + VIRTUAL_SHARES
         );
         if (shares > uint256(position.borrowShares)) {
             repayInAmounts = false;
-            shares = uint256(position.borrowShares);
+            shares = uint256(position.borrowShares) / 2; // MUTATION: full
repayment leaves half the shares behind
         } else {
             repayInAmounts = true;
         }

         if (repayInAmounts) {
             IMorpho(MARKET).repay(MARKET_PARAMS, amounts, 0, address(this), "");
         } else {
             IMorpho(MARKET).repay(MARKET_PARAMS, 0, shares, address(this), "");
```

Property source (`certora/specs/MorphoLoopPool.spec`)

```
/// MORPHO-RP-01: _repayDebt never tries to repay more shares than the pool
/// holds: repaying at least the (rounded-up) outstanding debt clears the
/// position, and repaying less reduces it.
rule repayDebtClearsOrReduces(env e, uint256 amount) {
    setup(e);
    mathint owed = systemDebt();
    mathint bs0 = to_mathint(morpho.borrowSharesOf(currentContract));
    repayDebtExt(e, amount);
    mathint bs1 = to_mathint(morpho.borrowSharesOf(currentContract));
    assert bs1 <= bs0;
    assert to_mathint(amount) >= owed => bs1 == 0;
}
```

Reproduce

```
# clean src/ -> VERIFIED
certoraRun certora/confs/mutation/MorphoLoopPool.conf --rule
repayDebtClearsOrReduces
# with the mutation -> rule VIOLATED = mutant caught
certoraMutate certora/confs/mutation/MorphoLoopPool.conf
```

○ MathLib.m1 — mulDivUp: `floor + 1` instead of a true ceiling

Mutant file	`certora/mutations/MathLib/MathLib.m1.sol`
Mutated source	`src/libraries/MathLib.sol`
Bug class	Off-by-one ceiling: returns floor+1 even when the division is exact.
Expected catcher	`mulDivUpIsCeil`
Status	not run (prover budget exhausted, see section 4)

Why this rule catches it. SL-08 asserts `(r−1)·d < x·y` for the result `r` of `mulDivUp`; when `d` divides `x·y` exactly the mutant returns one too many and the lower bound fails.

Diff against the audited source

```
--- a/src/libraries/MathLib.sol
+++ b/src/libraries/MathLib.sol
@@ -24,11 +24,11 @@
         return (x * y) / d;
     }

     function mulDivUp(
         uint256 x,
         uint256 y,
         uint256 d
     ) internal pure returns (uint256) {
-        return (x * y + (d - 1)) / d;
+        return (x * y) / d + 1; // MUTATION: always adds one instead of ceiling
     }
 }
```

Property source (`certora/specs/SharesLib.spec`)

```
/// SL-08: mulDivUp is a true ceiling of x*y/d.
rule mulDivUpIsCeil(uint256 x, uint256 y, uint256 d) {
    require d > 0;
    mathint exact = x * y;
    uint256 r = mulDivUp(x, y, d);
    assert r * d >= exact;
    assert (r - 1) * d < exact || r == 0;
}
```

Reproduce

```
# clean src/ -> VERIFIED
certoraRun certora/confs/mutation/SharesLib.conf --rule mulDivUpIsCeil
```

```
# with the mutation -> rule VIOLATED = mutant caught
certoraMutate certora/confs/mutation/SharesLib.conf
```

○ SharesLib.m1 — toSharesDown: rounds up instead of down

Mutant file	`certora/mutations/SharesLib/SharesLib.m1.sol`
Mutated source	`src/libraries/SharesLib.sol`
Bug class	Rounding direction flipped in a share conversion.
Expected catcher	`roundTripAssetsDownNeverGains`
Status	not run (prover budget exhausted, see section 4)

Why this rule catches it. SL-06 asserts `toAssetsDown(toSharesDown(a)) <= a`; with `toSharesDown` rounding up the round trip can return more assets than deposited.

Diff against the audited source

```
--- a/src/libraries/SharesLib.sol
+++ b/src/libraries/SharesLib.sol
@@ -9,17 +9,17 @@
     function toSharesDown(
         uint256 assets,
         uint256 totalAssets,
         uint256 totalShares
     ) internal pure returns (uint256) {
         if (totalShares == 0) {
             return assets;
         }
-        return assets.mulDivDown(totalShares, totalAssets);
+        return assets.mulDivUp(totalShares, totalAssets); // MUTATION: rounds up
     }

     function toSharesUp(
         uint256 assets,
         uint256 totalAssets,
         uint256 totalShares
     ) internal pure returns (uint256) {
         if (totalShares == 0) {
```

Property source (`certora/specs/SharesLib.spec`)

```
/// SL-06: round trip assets -> shares(down) -> assets(down) never gives back more.
rule roundTripAssetsDownNeverGains(uint256 a, uint256 tA, uint256 tS) {
    require tS > 0 && tA > 0;
    uint256 s = toSharesDown(a, tA, tS);
    uint256 back = toAssetsDown(s, tS, tA);
    assert back <= a;
}
```

Reproduce

```
# clean src/ -> VERIFIED
certoraRun certora/confs/mutation/SharesLib.conf --rule
roundTripAssetsDownNeverGains
# with the mutation -> rule VIOLATED = mutant caught
certoraMutate certora/confs/mutation/SharesLib.conf
```

7. Setup and Execution

All `certoraRun` commands are executed from the repository root at the audit commit with the git submodules initialised (`lib/openzeppelin-contracts`, `lib/openzeppelin-contracts-upgradeable`, `lib/forge-std`). Tested on macOS 15 with Homebrew; the Linux equivalents are `apt install default-jre` and `pipx`.

```
git submodule update --init --recursive
brew install openjdk@21 && export PATH="/opt/homebrew/opt/openjdk@21/bin:$PATH"   #
JDK 21+
pip3 install certora-cli==8.19.2 solc-select
solc-select install 0.8.28 && solc-select use 0.8.28
export CERTORAKEY=<your_certora_api_key>            # free key:
https://www.certora.com/signup?plan=prover
```

Production runs (each prints a `https://prover.certora.com/output/...` link with the full trace of every rule, including counterexample call sequences for violated rules):

```
certoraRun certora/confs/SharesLib.conf
certoraRun certora/confs/AaveV3LoopPool-A.conf      # invariants, effects, LTV
certoraRun certora/confs/AaveV3LoopPool-B.conf      # parametric access-control /
isolation / state-machine rules
certoraRun certora/confs/MorphoLoopPool-all.conf
# single-job variants (may exceed the 2-hour global timeout on the free tier):
certoraRun certora/confs/AaveV3LoopPool.conf
certoraRun certora/confs/MorphoLoopPool.conf
```

Mutation runs (results appear on https://prover.certora.com/mutations):

```
certoraMutate certora/confs/mutation/SharesLib.conf
certoraMutate certora/confs/mutation/LoopPool-core.conf
certoraMutate certora/confs/mutation/AaveV3LoopPool.conf
certoraMutate certora/confs/mutation/MorphoLoopPool.conf
```

Regenerating this report after a run: `python3 audit/tools/collect_results.py && python3 audit/tools/build_report.py`.

File inventory:

```
certora/
├── confs/            SharesLib.conf, AaveV3LoopPool{,-A,-B}.conf, MorphoLoopPool{,-
all}.conf, mutation/*.conf
├── specs/            SharesLib.spec, LoopPoolCore.spec, AaveV3LoopPool.spec,
MorphoLoopPool.spec
```

```
├── harness/          SharesLibHarness.sol, AaveV3LoopPoolHarness.sol,
MorphoLoopPoolHarness.sol, mocks/
└── mutations/        <Contract>/<Contract>.mN.sol + .txt
audit/
├── Looputo-Formal-Verification-Report.{md,html,pdf}
├── data/             results.json (collected verdicts), mutations.json (mutant
catalogue), findings.md
└── tools/            collect_results.py, build_report.py
```

8. Resources

- Certora Prover documentation — CVL reference, tutorials and the mutation verifier
- Certora Prover on GitHub — the open-source prover, buildable for local execution
- OpenZeppelin Certora specifications — reference specs for the Ownable, Initializable and proxy primitives this protocol builds on
- Morpho Blue formal verification — the virtual-shares arithmetic mirrored by `MockMorpho`